Intro to AI – Practical Assignment 1

How to run

Run ***project\src\Env\Main.java*** and provide it with the path to the input file as the first argument.

Project Structure

- **\src\Env** contain the main setup, environment and Enums
- **\src\Graph** contain the Graph, State, Simulator
- **\src\Player** contain an abstract Player class and each player type
- **\src\Test** contain Javadocs of the input file and test generator

Heuristic evaluation function

I have used the weight of the Minimum Spanning Tree for auxiliary graph containing only the relevant vertices.

Said vertices are:

- The player's current location
- The location of packages which were not carried by now
- The destination of packages which the current player holds

The rationale behind this heuristic is that the shortest path to deliver all the packages is at least as costly as the MST, because we'll have to traverse every edge on it at least once.

Furthermore, this heuristic is admissible, as the MST is the shortest path to travel between all the points of interest, and in practicality we might travel on the same edge more than once or take a longer route due to fragile edges.

In the case where all the packages are delivered (or unable to be delivered due to being on two different connected components), the MST's will be a 1 vertex graph with MST weight of 0.

Programing Bonus 1 & 2

I have implemented an extended solution for both bonus 1 and bonus 2.

My implementation support an arbitrary amount of saboteur agents and normal agents.

In state.java, the function simulateAgentsStep handle this.

```
157
158  private int simulateAgentsStep(Graph childGraph, int edgeIndex){
159      List<Player> statePlayers = childGraph.getPlayers();
160      Player childPlayer = statePlayers.get(this.controlloedPlayer.getId());
161
162      int deltaTime = this.graph.getVertecies()[this.currentLocation].getNeighborWeight(edgeIndex);
163      int deltaScore = 0;
164
165      while(deltaTime > 0){
166
167          //update search agent player
168          deltaScore += childPlayer.emulateStep(edgeIndex);
169
170          for(int playerIndex=0; playerIndex < statePlayers.size(); playerIndex++){
171              //non-search agents step simulation
172              if(statePlayers.get(playerIndex) != childPlayer && !statePlayers.get(playerIndex).isSearchAgent())
173                  statePlayers.get(playerIndex).travers();
174          }
175
176          deltaTime--;
177      }
178
179      return deltaScore;
180  }
```

Theoretical bonus

What is the computational complexity of the MAPD Problem (single agent) ? Can you prove that it is NP-hard? Or is it in P? If the latter, can you design an algorithm that solves the problem in polynomial time?

Answer:

The single agent MAPD problem is NP-Hard.

We'll show a polynomial reduction from The Traveling Salesman to the single agent MAPD problem, and since TSP is NP-Hard, our problem is NP-Hard as well.

Reduction:

Each city from TSP will be a vertex in the MAPD problem.

Every vertex Vi will hold a Package Pi with both initial location and destination location of Vi.

Our agent will start from the same vertex (city) the traveling salesman was starting from.

The packages will all be spawned at time=0, and have a delivery time of infinity.

Weights will be the same as in TSP, and there will be no fragile edges.

Therefore, the agent will have to visit all vertices in some order – satisfying the TSP problem as well.